

Abacus, a Virtual Environment for Learning Spatial Programming

Category: Demo

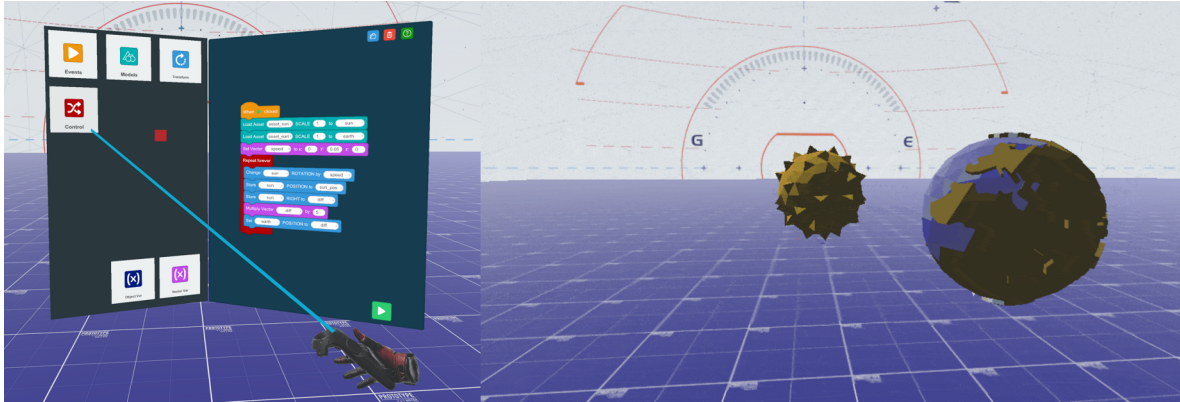


Figure 1: Abacus immersive programming mode. Left: programming interface showing block selection and coding panel. Right: solar system procedural animation.

ABSTRACT

Abacus is a virtual learning environment that combines the benefits of visual programming languages with the embodied advantages of virtual environments. As the number of applications involving 3D programming increases, immersive learning environments may help novice programmers achieve higher levels of understanding and performance.

Index Terms: education, k12, virtual reality, programming, blocks

1 INTRODUCTION

Various technologies have been developed to teach students CT skills in K-12 contexts. One of the most popular tools is Scratch, a block-based programming designed by MIT [8]. Block-based editors enable users to program by manipulating visual elements rather than typing text. They remove typing errors and the need to memorize the language structure with a palette of commands that can be dragged to create programs. Research has shown that students become more interested in computer science after working with a block-based interface, and that this approach can help with student retention [18, 6]. Block-based programming has been adopted to lower the barrier to programming across a variety of domains, including mobile development [10], art [1], databases [15], robots [17], and parallel programming [4].

There are few research prototypes built for immersive programming. My Reality (MYR) is an application that combines programming with a virtual reality visualization. [2] However, MYR requires users to use a code editor on a Web-Based platform outside of VR to write code. Abacus [13] is an XR-native programming environment that enables students to create procedural, interactive and dynamic scenes inside VR.

2 WHY PROGRAMMING IN VR

Many everyday uses of computing involve reasoning about space [3]. Programming plays a key role in designing and simulating three-dimensional structures in engineering, analyzing cities and natural environments in geographic information systems, and animating characters in films and television [9]. Spatial computation also underlies common technologies such as 3D video games, navigation and location-based services, and spatial user interfaces.

As technologies like augmented reality, 3D scanning, and 3D printing continue to expand, spatial computation has become increasingly common and woven into daily life.

Developing this category of software requires programmers to routinely use complex spatial operations such as changing between coordinate systems, manipulating transformation hierarchies, computing normals and projections, interpolating quaternions, converting between representations, etc. In more complex tasks, this involves conceptualizing spatial problems in terms of linear algebra operations and describing them in symbolic form (Figure 2).

A vector can be visualized as an arrow (with a size and direction), representing various embodied concepts such as velocity, acceleration, force, etc. A vector can also be represented symbolically by a column vector, an entity that can be manipulated as an object and also carries a process meaning (addition, multiplication, product). In the formal world, a vector is no longer a quantity with magnitude and direction. A vector is any element of a set over a field with defined operations of addition and multiplication, satisfying a list of axioms [16].

Mathematical development progresses from physical interaction with the world toward increasingly abstract forms of representation, enabling more powerful problem-solving. Spideh points out that while experts can move between worlds very naturally and rapidly, most students do not have the cognitive structure to perform the switch that is available to the expert [12]. Considering that extended reality is an inherently spatial medium, we hypothesize it can significantly improve the learning of spatial programming by easing the process of switching between representations and supporting spatial skills [14].

Recent work in psychology suggests that the way we think is influenced by how we perceive and move (sensorimotor contingencies). In fact, prior work has found that head-tracked, egocentric camera control in VR improves spatial memory, allowing better recall, increased confidence at visual search tasks, and memorization, when compared to a desktop [7, 5, 11].

3 ABACUS

Over the last few years, we have been developing Abacus, a VR learning environment that facilitates learning at the intersection of computational and spatial thinking [13]. It uses a block-based language inspired by Scratch, but unlike it, our application allows the

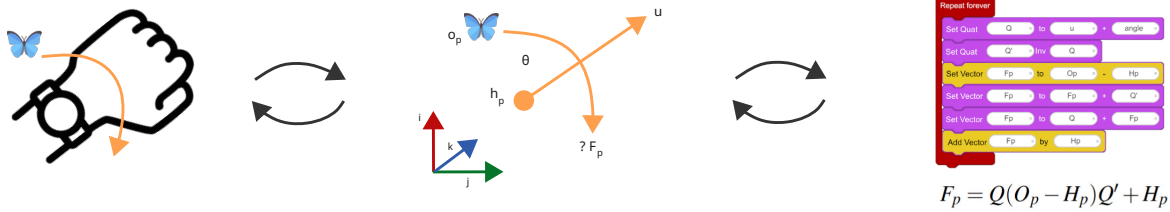


Figure 2: Example of a spatial programming task that students have difficulty with. How to move a butterfly around the wrist of the user? Left: Task representation; Middle: Embodied representation as geometric vectors and position in space; Right: symbolic representation as program instructions and linear algebra operations.



Figure 3: The selection buttons in both object variable and vector variables related blocks where the blocks require a selection of a variable (object variable or vector variable). Instead of a block slot, the selection button is placed, and the selection panel will show all the related variables for users to choose when clicked..

creation of procedural, interactive, and dynamic scenes inside VR (Figure 1).

The Abacus programming environment consists of: a) a block-based language interpreter, b) a set of 3D techniques for interaction, and c) UI interfaces for programming and auxiliary tasks. In desktop mode, users navigate using a first-person controller. The W, A, S, and D keys are used to move, the space bar to jump, and the mouse controls the camera orientation. In the VR mode, users can move around using natural walking or teleportation. A ray attached to the controller is used to interact with UI elements, including a virtual keyboard with a standard keyboard.

In Abacus, coding blocks are selected from a block palette panel. Navigation across different menu sections is simplified: users can click a section button to display the corresponding coding blocks and return to the main menu by selecting the main menu button. In contrast, Scratch differentiates values and variable types using distinct block slot shapes. However, Abacus supports a wider range of value types than Scratch, which is limited to only a few. To address this, instead of requiring users to drag specific values or variables into block slots, some slots are implemented as selection buttons. When clicked, these buttons open a selection panel that presents available options, allowing users to make selections without dragging blocks into the slots. One example of blocks with selection buttons are the Vector operation blocks (Figure 3).

There are twelve block sections available in Abacus:

- The Event section contains blocks that will respond to predefined events in Abacus. The event blocks are the entry points for their corresponding event. Thus, users can append coding scripts at the end of them to be triggered when the event is called by Abacus. This section is available in both the Abacus Creative mode and the Abacus Task mode.
- The Models section contains blocks that will help users create models. Currently, Abacus supports two ways to create models, loading existing models from the models asset folder and creating parametric primitive models.
- The Transform section contains blocks that manipulate the transformation (position, rotation, and direction) of the content of the object variable.
- The Control section contains blocks that have basic control over the execution flow of the coding script, including repeats (loops), breaks, waits, and if statements.
- the Interaction section contains blocks that can add interactions to the models stored in the object variable.
- The Visual section contains blocks that can change the visualization of the model stored in the object variable.
- The Sound section contains blocks that will play an audio clip (more advanced behavior-related blocks, such as audio control and spatial audio, are scheduled as part of future work).
- The Operators section contains blocks that provide arithmetic operations and comparison operations.
- The Logic Gates section contains blocks that provide logic gates for Boolean operations.
- The Value Variables section contains buttons, windows, and blocks that help users define and manipulate value variables (variables containing a number or string).
- The Object Variable section contains buttons that help users define object variables (variables that contain a model).
- The Vector Variable section contains buttons, windows, and blocks that help users define and manipulate vector variables (variables containing a vector).

For an initial evaluation, Sun and Lages recruited 40 university students, with 20 participants assigned to the desktop mode and 20

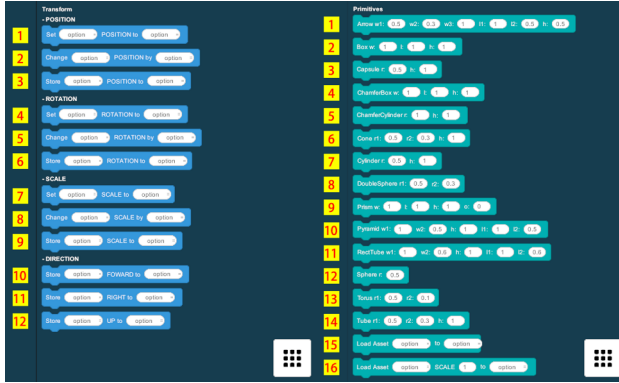


Figure 4: Transform and primitive block panels.

to the VR mode. Each session lasted 60 minutes, during which participants independently completed two programming tasks, a procedural task and a spatial task. The findings indicate that Abacus can improve student's self-efficacy, and that the gain increases as spatial abilities decrease. They have also evaluate the use of the environment for creative purposes and found that most participants in both studies showed a positive attitude towards using Abacus.

4 CONCLUSION

This describes a programming tools designed to better support learning in three-dimensional contexts. As applications of 3D programming continue to expand, such immersive programming environments hold promise for lowering barriers to entry and supporting more effective learning experiences for beginners.

REFERENCES

- [1] D. Bau, D. A. Bau, M. Dawson, and C. S. Pickens. Pencil code: block code for a text world. In *Proceedings of the 14th International Conference on Interaction Design and Children*, pp. 445–448, 2015. 1
- [2] C. Berns, G. Chin, J. Savitz, J. Kiesling, and F. Martin. Myr: A web-based platform for teaching coding using vr. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*, pp. 77–83, 2019. 1
- [3] A. DeHon, J.-L. Giavitto, and F. Gruau. 06361 executive report—computing media languages for space-oriented computation. In *Dagstuhl Seminar Proceedings*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2007. 1
- [4] A. Feng, M. Gardner, and W.-c. Feng. Parallel programming with pictures is a snap! *Journal of Parallel and Distributed Computing*, 105:150–162, 2017. 1
- [5] E. Krokos, C. Plaisant, and A. Varshney. Virtual memory palaces: immersion aids recall. *Virtual reality*, 23(1):1–15, 2019. 1
- [6] B. Moskal, D. Lurie, and S. Cooper. Evaluating the effectiveness of a new instructional approach. *ACM SIGCSE Bulletin*, 36(1):75–79, 2004. 1
- [7] R. Pausch, D. Proffitt, and G. Williams. Quantifying immersion in virtual reality. In *Proceedings of the 24th annual conference on Computer graphics and interactive techniques*, pp. 13–18. ACM Press/Addison-Wesley Publishing Co., 1997. 1
- [8] M. Resnick, J. Maloney, A. Monroy-Hernández, N. Rusk, E. Eastmond, K. Brennan, A. Millner, E. Rosenbaum, J. Silver, B. Silverman, and Y. Kafai. Scratch: programming for all. *Communications of the ACM*, 52(11):60–67, 2009. 1
- [9] S. Shekhar, S. K. Feiner, and W. G. Aref. Spatial computing. *Communications of the ACM*, 59(1):72–81, 2015. 1
- [10] W. Slany. Tinkering with pocket code, a scratch-like programming app for your smartphone. *Proceedings of Constructionism*, 2014. 1

- [11] S. A. Smith. Virtual reality in episodic memory research: A review. *Psychonomic bulletin & review*, 26(4):1213–1237, 2019. 1
- [12] S. Stewart. Moving between the embodied, symbolic and formal worlds of mathematical thinking with specific linear algebra tasks. In *Challenges and strategies in teaching linear algebra*, pp. 51–67. Springer, 2018. 1
- [13] N. Sun and W. S. Lages. Virtual reality impacts on novice programmers' self-efficacy. *IEEE Transactions on Visualization and Computer Graphics*, 2025. 1
- [14] D. H. Uttal, N. G. Meadow, E. Tipton, L. L. Hand, A. R. Alden, C. Warren, and N. S. Newcombe. The malleability of spatial skills: a meta-analysis of training studies. *Psychological bulletin*, 139(2):352, 2013. 1
- [15] R. Vinayakumar, K. Soman, and P. Menon. Db-learn: Studying relational algebra concepts by snapping blocks. In *2018 9th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, pp. 1–6. IEEE, 2018. 1
- [16] A. Watson, P. Spyrou, and D. Tall. The relationship between physical embodiment and mathematical symbolism: The concept of vector. *The Mediterranean Journal of Mathematics Education*, 1(2):73–97, 2003. 1
- [17] D. Weintrop, D. C. Shepherd, P. Francis, and D. Franklin. Blockly goes to work: Block-based programming for industrial robots. In *2017 IEEE Blocks and Beyond Workshop (B&B)*, pp. 29–36. IEEE, 2017. 1
- [18] D. Weintrop and U. Wilensky. Comparing block-based and text-based programming in high school computer science classrooms. *ACM Transactions on Computing Education (TOCE)*, 18(1):3, 2017. 1